

JavaScript Concepts

Rest parameters ...

Arrays:

1. Concatenation

```
let arr1 = [2, 3];
let arr2 = [1, ...arr1, 4];
console.log(arr2); // [1, 2, 3, 4]
```

```
let arr1 = [2, 3];
let arr2 = [4, 5];
let arr3 = [...arr1, ...arr2];

console.log(arr3); // [2, 3, 4, 5]
```

2. Creating Copies

```
let arr1 = [2, 3];
let arr2 = [...arr1];

console.log(arr2); // [2, 3]
```

Objects

```
let person = { name: "Rahul", age: 27 };
let personDetails = { ...person, city: "Hyderabad" };

console.log(personDetails); // Object {name: "Rahul", age: 27, city: "Hyderabad"}
```

Functions

A function can be called with a array of an argument, no matter how it is defined

```
function sumAll(...args) { // args is the name for the array
  let sum = 0;

  for (let arg of args) sum += arg;

  return sum;
}
```

```
alert( sumAll(1) ); // 1
alert( sumAll(1, 2) ); // 3
alert( sumAll(1, 2, 3) ); // 6
```

Features :

- Pack and Unpack arrays
- Dynamic number of parameters for a function

❶ Rest vs Arrays in Functions

Array: The function receives a **single argument**: the array itself.

Rest: The spread operator "**expands**" or "unpacks" an iterable (like an array) into its individual elements. When used in a function call, it passes each element as a separate argument

≡ Function designed to take array a parameter

```
function calculateSum(arr) {
  // arr is the whole array, e.g., [1, 2, 3]
  return arr.reduce((acc, curr) => acc + curr, 0);
}
const numbers = [1, 2, 3];
calculateSum(numbers); // returns 6
```

≡ Function Designed to use Rest Operator ...

```
function add(a, b, c) {
  // a, b, and c are individual arguments, e.g., 1, 2, 3
  return a + b + c;
}
const numbers = [1, 2, 3];
add(...numbers); // returns 6
```

❶ Rest Operator ... must be the last parameter of the function or a **SyntaxError** will occur

Rest `...` and Spread `...` Operator use similar syntax, but behaves differently based on the context. The rest operator **collects** multiple elements into an array, while the spread operator **expands** an iterable (like an array) into individual elements.

Ternary Operator

A Ternary Operator can be used to replace `if...else` statements in some situations.

Syntax: `condition ? expressionIfTrue : expressionIfFalse`

```
let speed = 70;
let message = speed >= 100 ? "Too Fast" : "OK";

console.log(message); // OK
```

Classes

Basic Syntax:

```
class MyClass {
  // class methods
  constructor() { ... }
  method1() { ... }
  method2() { ... }
  method3() { ... }
  ...
}
```

Then use `new MyClass()` to create a new object with all the listed methods.

The `constructor()` method is called automatically by `new`, so we can initialize the object there.

Example

```
class User {
  constructor(name) {
    this.name = name;
  }
  sayHi() {
    alert(this.name);
  }
}
// Usage:
let user = new User("John");
user.sayHi();
```

Prototype

```
class User {
  constructor(name) { this.name = name; }
  sayHi() { alert(this.name); }
}

// class is a function
console.log(typeof User); // function

// ...or, more precisely, the constructor method
console.log(User === User.prototype.constructor); // true

// The methods are in User.prototype, e.g:
console.log(User.prototype.sayHi); // the code of the sayHi method

// there are exactly two methods in the prototype
console.log(Object.getOwnPropertyNames(User.prototype)); // constructor, sayHi
```

Class inheritance

Class inheritance is a way for one class to extend another class, so we can create new functionality on top of the existing

≡ Example

```
class Animal {
  constructor(name) {
    this.speed = 0;
    this.name = name;
  }
  run(speed) {
    this.speed = speed;
    console.log(`${this.name} runs with speed ${this.speed}.`);
  }
  stop() {
    this.speed = 0;
    console.log(`${this.name} stands still.`);
  }
}

let animal = new Animal("My animal");
```

```
class Rabbit extends Animal {
  hide() {
    console.log(`${this.name} hides!`);
  }
}
```

```
}

let rabbit = new Rabbit("White Rabbit");

rabbit.run(5); // White Rabbit runs with speed 5.
rabbit.hide(); // White Rabbit hides!
```

Method Overriding

⋮ **stop()** is overridden

```
class Animal {

  constructor(name) {
    this.speed = 0;
    this.name = name;
  }

  run(speed) {
    this.speed = speed;
    alert(`${this.name} runs with speed ${this.speed}`);
  }

  stop() {
    this.speed = 0;
    alert(`${this.name} stands still.`);
  }

}

class Rabbit extends Animal {
  hide() {
    alert(`${this.name} hides!`);
  }

  stop() {
    super.stop(); // call parent stop
    this.hide(); // and then hide
  }
}

let rabbit = new Rabbit("White Rabbit");

rabbit.run(5); // White Rabbit runs with speed 5.
rabbit.stop(); // White Rabbit stands still. White Rabbit hides!
```

Sync and Async

Synchronous Functions

Code executes one after the other

```
console.log("Line 1")
console.log("Line 2")
console.log("Line 3")
```

Asynchronous Functions

The word “async” before a function means one simple thing: a function always returns a promise. Other values are wrapped in a resolved promise automatically.

```
async function f() {
  return 1;
}

f().then(alert); // 1
```

Await

The keyword `await` makes JavaScript wait until that promise settles and returns its result.

```
async function f() {

  let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("done!"), 1000)
  });

  let result = await promise; // wait until the promise resolves (*)

  alert(result); // "done!"
}

f();
```

Promises

Promise is a way to handle Asynchronous operations.

A promise is an object that represents a result of operation that will be returned at some point in the future.

```
let promise = new Promise(function(resolve, reject) {  
    ...  
});
```

The `promise` object returned by the `new Promise` constructor has these internal properties:

- `state` — initially `"pending"`, then changes to either `"fulfilled"` when `resolve` is called or `"rejected"` when `reject` is called.
- `result` — initially `undefined`, then changes to `value` when `resolve(value)` is called or `error` when `reject(error)` is called.

When the executor obtains the result, be it soon or late, doesn't matter, it should call one of these callbacks:

- `resolve(value)` — if the job is finished successfully, with result `value`.
- `reject(error)` — if an error has occurred, `error` is the error object.

Promise Chaining

Example

```
new Promise(function(resolve, reject) {  
  
    setTimeout(() => resolve(1), 1000); // (*)  
  
}).then(function(result) { // (**)  
  
    alert(result); // 1  
    return result * 2;  
  
}).then(function(result) { // (***)  
  
    alert(result); // 2  
    return result * 2;  
  
}).then(function(result) {  
  
    alert(result); // 4  
    return result * 2;  
  
});
```

Here the flow is:

1. The initial promise resolves in 1 second (*),
2. Then the `.then` handler is called (**), which in turn creates a new promise (resolved with 2 value).
3. The next `then` (***) gets the result of the previous one, processes it (doubles) and passes it to the next handler.
4. ...and so on.

As the result is passed along the chain of handlers, we can see a sequence of `alert` calls: 1 → 2 → 4

The whole thing works, because every call to a `.then` returns a new promise, so that we can call the next `.then` on it.

When a handler returns a value, it becomes the result of that promise, so the next `.then` is called with it